

Introduction à la programmation Python Débutant par Florent

Forma2+

3 jours

Cadrage

- Public : débutants complets
- Environnement : Windows + VS Code
- Objectif : automatisation et traitement de données
- Données : CSV / Excel / API simple

Modalité pédagogique

- formation en direct animée par le formateur
- démonstrations live au tableau et dans VS Code
- pratique encadrée avec correction immédiate
- questions/réponses en continu pendant les ateliers

Objectifs de sortie

À la fin de la formation, chaque participant sait :

- exécuter un script Python propre
- lire/transformer des données CSV/Excel
- structurer typer des fonctions simples
- gérer les erreurs de base
- consommer une API simple

Format d'exercices

Chaque exercice suit la même structure :

- objectif fonctionnel clair
- I/O attendues (entrées/sorties)
- exemples d'exécution
- cas limites à gérer
- critère binaire : "ça marche / ça ne marche pas"

Discipline de rendu des exercices

- un dossier de rendu par journée : rendus/J1, rendus/J2, rendus/J3
- des noms de fichiers stables et imposés
- exécution en ligne de commande (`python fichier.py`)
- au moins 3 tests manuels documentés

Template de rendu (obligatoire) :

```
rendus/JX/  
  README_execution.md  
  tests_manuels.md  
  src/  
    ...scripts python...  
  output/  
    ...fichiers générés...
```


Jour 1

Installation + fondamentaux Python

Jour 1 - Matin

- pourquoi Python
- installation Python/VS Code
- interpréteur Python (REPL)
- premier script
- extension Python VS Code
- linting + formatting
- pip + venv

Pourquoi Python (angle entreprise)

- langage lisible
- moins de code pour résoudre un problème
- multi-usage (data, automation, API, web)
- énorme écosystème

Installation Python (Windows)

- installer Python 3.11+
- cocher Add python.exe to PATH

Vérifier :

```
python --version
```

```
pip --version
```

Installation VS Code

- installer VS Code
- créer un dossier projet (hello_world)
- créer app.py

REPL (interpréteur)

python

Exemples pratiques :

2 + 2

2 > 1

2 > 5

Expression et syntaxe

- expression = code qui produit une valeur
- syntaxe = grammaire

Exemple erreur syntaxe :

2 >

Premier script

```
print("Hello world")  
print("*" * 10)
```

Exécution :

```
python app.py
```


VS Code : extension Python

Installer :

- Python (Microsoft)
- Pylance (Microsoft)

Effets immédiats :

- autocomplétion
- diagnostics
- bouton Run

Linting en action

```
print "hello"
```

```
2 +
```

Le linter signale l'erreur avant exécution.

Formatage (PEP 8)

- activer Format on Save
- garder un style constant

Mauvais :

```
x=1  
unit_price      =3
```

Bon :

```
x = 1  
unit_price = 3
```

`pip` : **packages**

```
python -m pip install requests
```

```
python -m pip install pandas openpyxl
```

```
python -m pip list
```

```
python -m pip freeze > requirements.txt
```

venv : environnement isolé

```
python -m venv .venv
```

```
.\.venv\Scripts\activate
```

```
python -m pip install --upgrade pip
```

```
deactivate
```

Mini atelier matin (30 min)

À valider :

- `app.py` exécutable
- extension Python active
- venv créé/activé
- `requirements.txt` généré

Jour 1 - Après-midi

- variables et types
- strings (index/slice/méthodes)
- nombres et opérateurs
- input + conversion + truthy/falsy
- atelier nettoyage d'une ligne Excel

Variables et types primitifs

```
students_count = 1000  
rating = 4.99  
is_published = True  
course_name = "Python Programming"
```


Nommage propre

- noms descriptifs
- snake_case
- éviter noms cryptiques (c1, cn)
- espaces autour de =

Strings - longueur/index/slice

```
course = "Python Programming"
```

```
print(len(course))
```

```
print(course[0])
```

```
print(course[-1])
```

```
print(course[:3])
```

```
print(course[7:])
```


Strings - échappement

```
message = "Python \"Workshop\""  
path = "C:\\temp\\data"  
multiline = "line1\\nline2"
```

Strings - f-strings

```
first = "Alice"  
last = "Durand"  
full = f"{first} {last}"  
print(full)
```

Strings - méthodes utiles

```
course = " python programming "
```

```
print(course.strip())  
print(course.upper())  
print(course.lower())  
print(course.title())  
print(course.find("pro"))  
print(course.replace("p", "j"))  
print("python" in course)
```

Nombres et opérateurs

```
x = 10
```

```
print(x / 3)
```

```
print(x // 3)
```

```
print(x % 3)
```

```
print(x ** 3)
```

```
x += 3
```

```
print(x)
```

Fonctions numériques

```
import math
```

```
print(round(2.9))
```

```
print(abs(-2.9))
```

```
print(math.ceil(2.2))
```

input + conversion

```
x = input("x: ")  
y = int(x) + 1  
print(f"x={x}, y={y}")
```


Truthy / Falsy

Falsy :

- `0`
- `""`
- `None`

Tout le reste est truthy.

Exercices Jour 1

Objectif : ancrer les bases sur des cas de nettoyage de données.

Règle pédagogique : aucune notion non vue à ce stade.

Notions autorisées :

- variables
- `print / input`
- strings (`strip`, `lower`, `replace`, `in`)
- conversion (`float`)
- booléens (`truthy/falsy`)

Parcours :

1. nettoyer une cellule d'en-tête
2. nettoyer une cellule email
3. nettoyer une cellule montant

Exercice J1-A - Nettoyer une cellule d'en-tête

Spécification :

- entrée : `header_raw = " Nom Client "`
- sortie attendue : `nom_client`
- contrainte : une seule expression de nettoyage

Exemple :

- `Nom Client` -> `nom_client`
- `Montant` -> `montant`

Correction J1-A

```
header_raw = " Nom Client "  
header_clean = header_raw.strip().lower().replace(" ", "_")  
print(header_clean)
```

Exercice J1-B - Nettoyer une cellule email

- entrée : `email_raw` saisi au clavier
- appliquer :
 - `trim`
 - `lowercase`
- afficher :
 - email nettoyé
 - booléen de validité (présence de @)

Exemple :

- `Alice@ACME.COM` -> `alice@acme.com`
- `contact_acme.com` -> invalide

Correction J1-B

```
email_raw = input("Email: ")
email_clean = email_raw.strip().lower()
is_valid = "@" in email_clean

print("email_clean =", email_clean)
print("email_valide =", is_valid)
```

Exercice J1-C - Nettoyer une cellule montant

Spécification :

- entrée : raw = "1 200,50 €"
- appliquer :
 - trim
 - suppression espaces / €
 - remplacement , par .
- sortie :
 - montant texte nettoyé
 - montant converti en float

Exemple :

- 1 200,50 € -> "1200.50" puis 1200.5

Correction J1-C

```
raw = "1 200,50 €"
amount_text = raw.strip().replace("€", "").replace(" ", "").replace(",", ".", ".")
amount_value = float(amount_text)

print("montant_net =", amount_text)
print("type =", type(amount_value))
print("valeur =", amount_value)
```


Atelier Jour 1 (45 min) - Nettoyage d'une ligne Excel

Objectif : nettoyer une ligne saisie par l'utilisateur, sans boucle ni fonction perso.

Entrées :

- `header_raw`
- `email_raw`
- `amount_raw`

Sorties :

- `header_clean`
- `email_clean`
- `email_valide`
- `amount_net`

Squelette atelier J1

```
header_raw = input("Header: ")
email_raw = input("Email: ")
amount_raw = input("Montant: ")

header_clean = header_raw.strip().lower().replace(" ", "_")
email_clean = email_raw.strip().lower()
email_valide = "@" in email_clean
amount_net = amount_raw.strip().replace("€", "").replace(" ", "").replace(",", ".", ".")

print("header_clean =", header_clean)
print("email_clean =", email_clean)
print("email_valide =", email_valide)
print("amount_net =", amount_net)
print("amount_float =", float(amount_net))
```


Livrable Jour 1

Dossier attendu : `rendus/J1/`

- `rendus/J1/src/j1_exo_a.py`
- `rendus/J1/src/j1_exo_b.py`
- `rendus/J1/src/j1_exo_c.py`
- `rendus/J1/src/atelier_j1_ligne_excel.py`
- `rendus/J1/README_execution.md` (commande de lancement)
- `rendus/J1/tests_manuels.md` (3 cas minimum)
- `rendus/J1/output/` (captures ou sorties texte)

Validation :

- les 4 scripts s'exécutent sans erreur
- les sorties affichées correspondent à la spécification

Quiz flash Jour 1

1. Différence REPL vs script ?
2. Pourquoi venv ?
3. Quand convertir `str` en `int/float` ?

Jour 2

**Logique Python + boucles +
fonctions**

Jour 2 - Matin

- comparaisons
- conditions
- opérateurs logiques
- boucles for/while
- exercices guidés

Comparaisons

```
print(10 > 3)
```

```
print(10 >= 3)
```

```
print(10 == 10)
```

```
print(10 != "10")
```

if / elif / else

```
temperature = 35
```

```
if temperature > 30:  
    print("It is warm")  
elif temperature > 20:  
    print("It is nice")  
else:  
    print("It is cold")
```

```
print("Done")
```


Ternaire

```
age = 22  
message = "Eligible" if age >= 18 else "Not eligible"  
print(message)
```

and / or / not

```
high_income = True  
good_credit = False  
student = False
```

```
eligible = (high_income or good_credit) and not student  
print(eligible)
```


Short-circuit

- and coupe au premier False
- or coupe au premier True

Comparaison chaînée

```
age = 22
```

```
if 18 <= age < 65:  
    print("eligible")
```

Boucle for + range

```
for number in range(1, 4):  
    print("Attempt", number)  
    print("." * number)
```

for + break + else

```
successful = False
```

```
for attempt in range(1, 4):  
    print("Attempt", attempt)  
    if successful:  
        print("Successful")  
        break  
else:  
    print("Attempted 3 times and failed")
```

Boucles imbriquées

```
for x in range(3):  
    for y in range(2):  
        print(f"({x}, {y})")
```

while (pattern interactif)

```
while True:
    command = input("> ")
    if command.lower() == "quit":
        break
    print("echo", command)
```


Structure de données utilisée dans les exos J2

- une ligne métier = un dictionnaire
- plusieurs lignes = une liste de dictionnaires
- on lit/écrit avec `row["colonne"]`

```
rows = [  
    {"email": "alice@acme.com", "montant": "1200.50"},  
    {"email": "bob@example.fr", "montant": "980.00"},  
]
```

```
for row in rows:  
    print(row["email"], row["montant"])
```

Exercices guidés Jour 2

Objectif : nettoyage CSV + logique métier.

Règle pédagogique :

- J2-A/J2-B/J2-C : uniquement boucles + conditions
- J2-D : première utilisation de fonctions personnalisées

Parcours :

1. normaliser les montants
2. dédupliquer les clients
3. produire des KPI fiables

Exercice J2-A - Normaliser la colonne montant

Spécification :

- convertir en float des valeurs hétérogènes :
 - 1 200,50 €
 - 1200.50
 - 1,200.50 (format us)
- sortir montant_net

Exemples :

- 1 200,50 € -> 1200.50
- N/A -> valeur invalide à tracer

Correction J2-A

```
raw_values = ["1 200,50 €", "1200.50", "1,200.50", "N/A"]  
normalized = []
```

```
for raw in raw_values:  
    s = raw.strip().replace("€", "").replace(" ", "")
```

```
    if s.lower() == "n/a" or s == "":
```

```
        normalized.append(None)
```

```
    elif "," in s and "." in s:
```

```
        # cas us: 1,200.50
```

```
        normalized.append(float(s.replace(",", "", "")))
```

```
    elif "," in s:
```

```
        # cas fr: 1200,50
```

```
        normalized.append(float(s.replace(",", "", ".")))
```

```
    else:
```

```
        normalized.append(float(s))
```

```
print(normalized)
```

Exercice J2-B - Dédupliquer par email

Spécification :

- nettoyer email
- supprimer doublons sur email
- garder la ligne la plus récente (date_maj)

Compétences : tri, conditions, qualité de données.

Correction J2-B

```
rows = [  
    {"nom": "Alice", "email": "alice@acme.com", "date_maj": "2026-03-01"},  
    {"nom": "Alice B", "email": " Alice@ACME.COM ", "date_maj": "2026-03-05"},  
    {"nom": "Bob", "email": "bob@example.fr", "date_maj": "2026-03-02"},  
]
```

```
dedup = {}
```

```
for row in rows:  
    email = row["email"].strip().lower()  
    row["email_clean"] = email  
  
    if email not in dedup or row["date_maj"] > dedup[email]["date_maj"]:  
        dedup[email] = row
```

```
result = list(dedup.values())  
print(result)
```

Exercice J2-C - KPI de qualité de données

Spécification :

- à partir d'un CSV nettoyé, calculer :
 - nb lignes totales
 - nb lignes valides
 - nb anomalies
 - CA total valide

Compétences : boucles + conditions + agrégation.

Correction J2-C

```
rows = [
    {"email_valide": "true", "montant_net": "1200.50"},
    {"email_valide": "false", "montant_net": "980.00"},
    {"email_valide": "true", "montant_net": ""},
    {"email_valide": "true", "montant_net": "450.00"},
]

total = len(rows)
valid = 0
ca = 0.0

for row in rows:
    email_ok = row["email_valide"].lower() == "true"
    amount_ok = row["montant_net"] != ""

    if email_ok and amount_ok:
        valid += 1
        ca += float(row["montant_net"])

anomalies = total - valid
print({"total": total, "valid": valid, "anomalies": anomalies, "ca": ca})
```


Jour 2 - Après-midi

- fonctions
- paramètres vs arguments
- return
- arguments nommés / défaut / *args
- point d'entrée main et `if __name__ == "__main__"`
- map, filter, sum
- typage
- atelier CSV

Function simple

```
def greet(first_name: str, last_name: str) -> None:  
    print(f"Hi {first_name} {last_name}")
```

```
greet("John", "Smith")
```


Paramètre vs argument

```
def greet(name: str): # paramètre  
    print(f"Hi {name}")
```

```
greet("Alice") # argument
```

Fonction qui retourne

```
def get_greeting(name: str) -> str:  
    return f"Hi {name}"
```

```
message = get_greeting("Alice")  
print(message)
```

Keyword args et défaut

```
def increment(number: int, by: int = 1) -> int:  
    return number + by
```

```
print(increment(2))  
print(increment(2, by=5))
```

***args**

```
def multiply(*numbers: int) -> int:  
    total = 1  
    for number in numbers:  
        total *= number  
    return total
```

```
print(multiply(2, 3, 4))
```

Point d'entrée de script (__main__)

```
def main() -> None:  
    print("Traitement J2")
```

```
if __name__ == "__main__":  
    main()
```

map / filter / sum

```
rows = [  
    {"email_valide": "true", "montant_net": "1200.50"},  
    {"email_valide": "false", "montant_net": "980.00"},  
    {"email_valide": "true", "montant_net": "450.00"},  
]  
  
valid_rows = list(filter(lambda r: r["email_valide"] == "true", rows))  
amounts = list(map(lambda r: float(r["montant_net"]), valid_rows))  
ca = sum(amounts)  
  
print("valid=", len(valid_rows))  
print("ca=", ca)
```


map / filter / sum vs boucle for

- map : transformer chaque élément
- filter : garder seulement les éléments voulus
- sum : agréger des nombres
- en débutant : garder la version for si c'est plus lisible

Typage attendu

- typer les signatures de fonctions
- typer les structures principales
- garder le code lisible et explicite

Exercice J2-D - Fonctions testables

Spécification :

- écrire `normalize_amount(raw: str) -> float | None`
- écrire `is_valid_email(email: str) -> bool`
- tester manuellement 3 cas par fonction (normal, bord, invalide)

Correction J2-D

```
def normalize_amount(raw: str) -> float | None:
    s = (raw or "").strip().lower().replace(" ", "").replace("€", "")
    if s in {"", "n/a", "na"}:
        return None
    if "," in s and "." not in s:
        s = s.replace(",", ".")
    elif "," in s and "." in s:
        s = s.replace(",", "")
    try:
        return float(s)
    except ValueError:
        return None
```

```
def is_valid_email(email: str) -> bool:
    value = (email or "").strip().lower()
    return "@" in value and "." in value
```

```
assert normalize_amount("1 200,50 €") == 1200.5
assert normalize_amount("N/A") is None
assert normalize_amount("") is None
```

```
assert is_valid_email(" Alice@acme.com ") is True
assert is_valid_email("contact_acme.com") is False
assert is_valid_email("") is False
```

```
print("tests manuels: OK")
```

Atelier Jour 2 (90 min)

Pipeline métier :

1. lire CSV
2. nettoyer données
3. calcul KPI simples
4. exporter CSV

Exemple CSV (DictReader)

```
import csv

with open("entree.csv", newline="", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))
```

Livrable Jour 2

Dossier attendu : `rendus/J2/`

- `rendus/J2/src/j2_exo_a.py`
- `rendus/J2/src/j2_exo_b.py`
- `rendus/J2/src/j2_exo_c.py`
- `rendus/J2/src/j2_exo_d.py`
- `rendus/J2/src/script_j2_traitement.py`
- `rendus/J2/output/sortie_j2.csv`
- `rendus/J2/README_execution.md`
- `rendus/J2/tests_manuels.md`

Validation :

- pipeline complet exécutable (entrée -> sortie)
- point d'entrée clair via `if __name__ == "__main__":`
- résultats cohérents sur les cas de test fournis

Quiz flash Jour 2

1. À quoi sert `if __name__ == "__main__":` ?
2. Différence entre `map` et `filter` ?
3. Quand garder une boucle `for` plutôt que `map/filter` ?

Jour 3

**CSV -> Gemini Assist -> API +
robustesse + mini-projet final**

Jour 3 - Matin

- lire un CSV métier
- transformer en ajoutant des colonnes
- écrire un nouveau CSV transformé
- configuration Gemini Assist dans VS Code
- workflow entreprise (code métier + contraintes)
- API : principe
- GET + token
- JSON -> CSV
- pandas + Excel
- validations minimales

J3 - Bloc CSV de démarrage

Objectif :

- partir d'un CSV brut
- ajouter des colonnes dérivées
- produire un CSV de sortie exploitable

Lire un CSV (DictReader)

```
import csv

with open("input/commandes_input_exemple.csv", newline="", encoding="utf-8") as f:
    rows = list(csv.DictReader(f))

print("nb_lignes =", len(rows))
print("colonnes =", list(rows[0].keys()))
```

Transformer : ajouter des colonnes

```
for row in rows:
    email_clean = row["email"].strip().lower()
    row["email_clean"] = email_clean
    row["email_valide"] = "@" in email_clean and "." in email_clean

    amount_text = row["montant"].strip().replace("€", "").replace(" ", "").replace(",", ".", ".")
    row["montant_net"] = amount_text
```

Réécrire dans un nouveau CSV

```
fieldnames = list(rows[0].keys())

with open("output/commandes_transformees.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=fieldnames)
    writer.writeheader()
    writer.writerows(rows)

print("OK -> output/commandes_transformees.csv")
```

Exercice J3-00 - CSV

transformation de base

Spécification :

- lire `input/commandes_input_exemple.csv`
- ajouter `email_clean`, `email_valide`, `montant_net`
- écrire `output/commandes_transformees.csv`

Contrainte :

- ne pas modifier/supprimer les colonnes d'origine

Correction J3-00

Validation minimale :

- fichier de sortie créé
- mêmes lignes que l'entrée
- colonnes d'origine conservées
- nouvelles colonnes bien remplies

J3 - Bloc Gemini Assist entreprise

Objectif :

- cadrer l'usage de Gemini sur code métier
- éviter toute exposition des documents confidentiels
- rendre le workflow reproductible en équipe

Workspace cible (business_logic/ script1)

```
business_logic/  
  libs/  
    cleaners.py  
    csv_io.py  
script1/  
  main.py  
  input/  
    commandes_input_exemple.csv  
  output/  
  secret/  
    MODELE_fichiers_secret.md  
  .aiexclude  
  .gitignore
```


Règle sécurité (non négociable)

- `secret/` ne doit pas entrer dans le contexte Gemini
- prompts basés sur `main.py` + `libs/` + `input/` uniquement
- les données d'entrée sont fausses mais structurellement réalistes
- les vrais documents restent dans `secret/` (usage runtime local)

Architecture code (non négociable)

- `business_logic/libs/` = fonctions génériques et réutilisables
- une responsabilité par fichier
- aucune logique métier spécifique dans `libs/`
- la logique métier locale peut rester dans `main.py`
- toute utilité générique va dans `libs/`

Fichier .aiexclude attendu

```
secret/  
output/  
.env  
*.key  
*.pem
```

Checklist config Gemini dans VS Code

1. ouvrir `business_logic/script1` comme racine de workspace
2. activer Gemini Assist avec le compte entreprise
3. vérifier que `.aiexclude` est présent en racine
4. garder l'exclusion `.gitignore` active
5. vérifier dans le Context Drawer que `secret/` n'est pas sélectionné

Workflow métier avec contrainte entreprise

1. mettre à jour `input/commandes_input_exemple.csv` (fausses données)
2. demander à Gemini une modif sur `libs/*.py` (générique) ou `main.py` (métier local)
3. exécuter localement et valider `output/resultat_exemple.csv`
4. seulement ensuite, exécuter en environnement entreprise avec `secret/`

Template de prompt équipe

Support à fournir aux étudiants :

- `business_logic/script1/PROMPT_TEMPLATE.md`
- même structure de prompt pour toute l'équipe
- contrôle systématique : pas de `secret/` dans la demande

Exercice J3-0 - Setup Gemini entreprise

Spécification :

- configurer Gemini dans VS Code sur `business_logic/script1`
- valider que `secret/` est exclu
- valider que `libs/` reste générique
- lancer `main.py` sur input d'exemple

Livrables :

- capture de la config effective
- `output/resultat_exemple.csv`
- court compte-rendu : ce qui est autorise / interdit

Correction J3-0

Checklist de validation :

- `.aiexclude` contient bien `secret/`
- aucun fichier de `secret/` dans le contexte Gemini
- script exécuté avec succès sur `input/`
`commandes_input_exemple.csv`

API : principe

- endpoint
- auth token
- réponse JSON
- mapping vers format métier

Requête GET simple

```
import os
import requests

url = "https://api.example.com/orders"
token = os.getenv("API_TOKEN", "")
headers = {"Authorization": f"Bearer {token}"}

r = requests.get(url, headers=headers, timeout=20)
r.raise_for_status()
data = r.json()
```

Export JSON vers CSV

```
import csv

rows = []
for item in data:
    rows.append({
        "id": item.get("id"),
        "client": item.get("client"),
        "montant": item.get("amount", 0),
    })

with open("api_orders.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["id", "client", "montant"])
    writer.writeheader()
    writer.writerows(rows)
```

Lecture Excel avec pandas

```
import pandas as pd
```

```
df = pd.read_excel("entree.xlsx")  
print(df.head(3))
```

Nettoyage et export Excel avec pandas

```
import pandas as pd

df = pd.read_excel("entree.xlsx")
df["email"] = df["email"].astype(str).str.strip().str.lower()
df["montant"] = df["montant"].astype(str).str.replace("€", "", regex=False)
df.to_excel("sortie.xlsx", index=False)
```

Jour 3 - Après-midi

- try/except
- logging
- argparse
- mini-projet
- restitution

Gestion d'erreurs

```
import logging
import requests

logging.basicConfig(level=logging.INFO)

try:
    r = requests.get(url, headers=headers, timeout=20)
    r.raise_for_status()
except requests.RequestException as e:
    logging.error("Echec API: %s", e)
    raise SystemExit(1)
```

Paramétrage CLI

```
import argparse
```

```
parser = argparse.ArgumentParser()  
parser.add_argument("--input", required=True)  
parser.add_argument("--output", required=True)  
args = parser.parse_args()
```


Exercice J3-A - Compter les lignes utiles

Spécification :

- appeler un endpoint GET `/orders`
- sauvegarder la réponse brute dans `orders_raw.json`
- tracer code HTTP et nombre d'objets reçus

Compétences : requests + fichiers JSON + logging.

Correction J3-A

```
import json
import logging
import requests

logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")

url = "https://api.example.com/orders"
r = requests.get(url, timeout=20)
r.raise_for_status()
orders = r.json()

with open("orders_raw.json", "w", encoding="utf-8") as f:
    json.dump(orders, f, ensure_ascii=False, indent=2)

logging.info("http_status=%s nb_orders=%s", r.status_code, len(orders))
```

Exercice J3-B - Mapper JSON API vers CSV métier

Spécification :

- mapper les champs API vers schéma cible :
 - `id` -> `commande_id`
 - `client` -> `client`
 - `amount` -> `montant`
- exporter `orders_clean.csv`

Compétences : transformation + export CSV.

Correction J3-B

```
import csv
import json

with open("orders_raw.json", encoding="utf-8") as f:
    data = json.load(f)

with open("orders_clean.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["commande_id", "client", "montant"])
    writer.writeheader()

    for item in data:
        writer.writerow(
            {
                "commande_id": item.get("id"),
                "client": item.get("client"),
                "montant": item.get("amount"),
            }
        )
```

Exercice J3-C - Fiabiliser la requête API

Spécification :

- gérer `timeout`, erreurs réseau et code HTTP non 200
- prévoir un fallback JSON local (`orders_fallback.json`)
- journaliser les erreurs dans un fichier log

Compétences : robustesse d'intégration API.

Correction J3-C

```
import json
import logging
from pathlib import Path

import requests

URL = "https://api.exemple.com/orders"
FALLBACK = Path("orders_fallback.json")

logging.basicConfig(
    filename="api_errors.log",
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s",
)

def load_orders() -> list[dict]:
    try:
        r = requests.get(URL, timeout=20)
        r.raise_for_status()
        data = r.json()
        if not isinstance(data, list):
            raise ValueError("Format JSON inattendu")
        return data
    except (requests.RequestException, ValueError) as exc:
        logging.exception("API indisponible, fallback local: %s", exc)
        if not FALLBACK.exists():
            raise SystemExit("Fallback manquant: orders_fallback.json")
        return json.loads(FALLBACK.read_text(encoding="utf-8"))

orders = load_orders()
print(f"orders={len(orders)}")
```


Mini-projet final (progressif)

Objectif métier unique :

- transformer `exercices/J3_Mini_projet/donnees_exemple.csv` en `exercices/J3_Mini_projet/resultat_exemple.csv`
- nettoyer email et montant
- marquer les anomalies

Règle :

- 3 paliers alignés avec le cours
- un palier validé avant de passer au suivant

Palier 1 (niveau Jour 1)

Notions autorisées :

- `input / print`
- `strings (strip, lower, replace, in)`
- `conversion float`

Tâche :

- nettoyer une seule ligne saisie au clavier
- afficher `email_clean`, `email_valide`, `montant_net`

Livrable :

- `mini_p1.py`

Palier 2 (niveau Jour 2)

Notions autorisées :

- `for`, `if`, structures `list/dict`
- fonctions simples
- module `csv`

Tâche :

- lire `exercices/J3_Mini_projet/donnees_exemple.csv`
- nettoyer toutes les lignes
- écrire `exercices/J3_Mini_projet/resultat_intermediaire.csv`

Livrables :

- `mini_p2.py`
- `exercices/J3_Mini_projet/resultat_intermediaire.csv`

Palier 3 (niveau Jour 3)

Notions autorisées :

- `requests, json`
- `try/except`
- `logging`

Tâche :

- enrichir ou contrôler les données via API
- gérer échec API avec fallback local
- générer `exercices/J3_Mini_projet/resultat_exemple.csv`

Livrables :

- `rendus/J3/src/script_final.py`
- `rendus/J3/output/resultat_exemple.csv`
- `rendus/J3/README_execution.md`
- `rendus/J3/tests_manuels.md`

Modes dégradés (pilotage formateur)

- si API KO : fallback local JSON obligatoire
- si palier 3 bloque : validation possible au palier 2
- si retard : correction guidée sur palier 3

Évaluation finale (par palier)

Atteint / non atteint :

- Palier 1 : nettoyage correct d'une ligne
- Palier 2 : pipeline CSV exécutable bout en bout
- Palier 3 : robustesse API + logs + fallback
- Qualité globale : lisibilité + nommage + README exploitable

Bonus - Transformer un script Python en .exe

Objectif :

- distribuer un script Python à un utilisateur Windows sans lui faire lancer `python mon_script.py`
- générer un exécutable à partir d'un script existant
- comprendre les limites réelles du packaging

Bonus - Pré-requis

- environnement Python propre et fonctionnel
- script exécutable en ligne de commande avant packaging
- dépendances installées dans l'environnement courant
- build .exe à faire sur Windows

Règle :

- si le script ne marche pas en .py, il ne marchera pas mieux en .exe

Bonus - Installer PyInstaller

Commande :

```
pip install pyinstaller
```

Vérification :

```
pyinstaller --version
```

Bonus - Commande minimale

Exemple :

```
pyinstaller --onefile main.py
```

Résultat attendu :

- un exécutable unique dans `dist/`
- des fichiers de build dans `build/`
- un fichier `main.spec`

Bonus - Exemple appliqué au cours

Depuis le dossier du script :

```
cd business_logic/script1  
pyinstaller --onefile --name transformateur_csv main.py
```

Exécution du binaire généré sous Windows :

```
dist\\transformateur_csv.exe --input input\\commandes_input_exemple.csv --output output\\resultat_exemple.csv
```

Bonus - Options utiles

- `--onefile` : un seul exécutable
- `--name mon_outil` : nommer le binaire
- `--windowed` ou `--noconsole` : masquer la console pour une app graphique

Exemple :

```
pyinstaller --onefile --name mon_outil main.py
```

Bonus - Ce que PyInstaller génère

- `dist/` : livrable final
- `build/` : fichiers temporaires de construction
- `mon_script.spec` : configuration du packaging

À ignorer dans Git :

- `build/`
- `dist/`
- `*.spec` si vous ne souhaitez pas versionner la config de build

Bonus - Limites importantes

- pas de cross-compilation simple : pour faire un `.exe`, build sur Windows
- certains antivirus peuvent ralentir ou bloquer un binaire — `onefile`
- les chemins de fichiers relatifs doivent être testés après packaging
- un `.exe` n'est pas une sécurité : le code reste distribuable, pas secret

Bonus - Workflow propre

1. valider le script Python en CLI
2. figer les dépendances
3. lancer `pyinstaller`
4. tester le `.exe` sur un poste Windows propre
5. documenter la commande d'exécution et les fichiers attendus

À ne pas faire :

- packager trop tôt
- mélanger debug du script et debug du packaging
- embarquer des fichiers sensibles dans le binaire

[fit] Fin de formation